

COMP7710

Programming Fundamentals

Today's lecture
Object-Oriented Programming (Advanced)

Course Convenor
Bernardo Pereira Nunes



Australian
National
University

All lectures are recorded and made available on Canvas

Agenda

- > Intro to Inheritance
- > Classes, Superclasses, and Subclasses
- > Polymorphism
- > Superclass (equals, hashCode, ...)
- > Generic ArrayList
- > Wrappers and Autoboxing
- > Abstract Classes
- > Design hints for inheritance
- > Exercises



Inheritance

In everyday English, inheritance refers to receiving something (for free) without having to create it yourself.

in java/OO

The idea behind inheritance is to **create new classes based on existing ones**. By inheriting from a class, you **reuse its methods and can add new methods and fields** to adapt the class to new situations. Inheritance is a fundamental concept in OO programming.



Philosophically...

- > ...inheritance comes from a desire to model the world through classification and reuse
- > Object-oriented thinking emerged from the observation that humans understand complex systems by **grouping things into categories** and then refining them
- > We say “this is a kind of that”: a *student* is a kind of *person*; a *car* is a kind of *vehicle*
- > These categories share common properties, but each refinement adds specificity
- > Inheritance formalizes this way of thinking in code



Categorization

> Categorization is the process of grouping ideas or objects using some purpose or relationship between them

How do you categorize things? Have you ever thought about how you do it?

> Usually, people categorize “things” in term of prototypes

>> The prototype theory was initially known as the concept of basic-level categorization

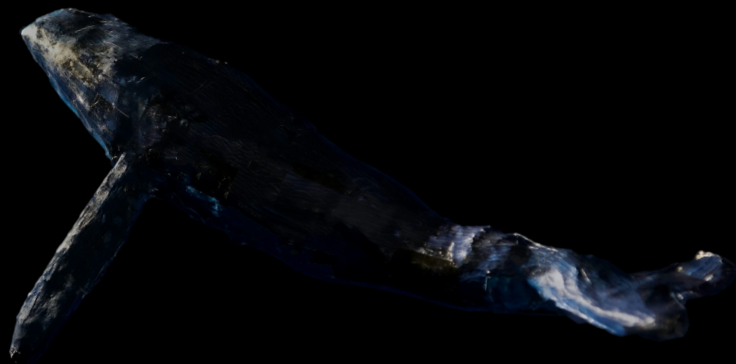
>> What would be the basic-level categorization of concepts such as “furniture” or “animal”?

>>> It could be a “chair” (not a stool) or a “robin,” but not a duck

>> For each concept, the basic-level can be more specific or more general



Categorization

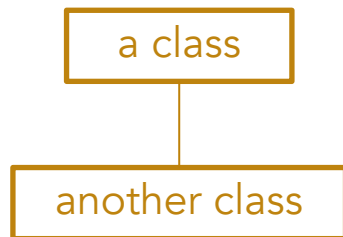


Classes, Superclasses, and Subclasses

- > Classes can be derived from other classes
- >> the derived class is called a subclass, and the class it is derived from is called the superclass
- > In Java, all classes must be derived from some class; the root class is `java.lang.Object`

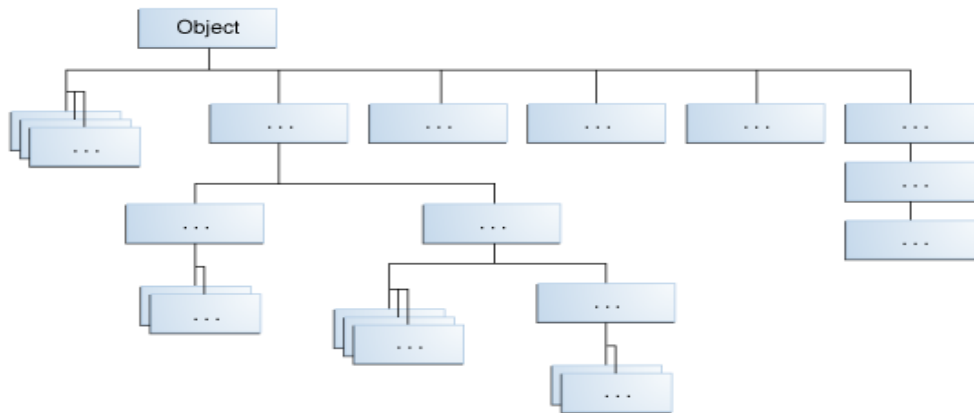
Superclass

(base class, ancestor
class or a parent class)



Subclass

(derived class, extended
class, or child class)



All Classes in the Java Platform are Descendants of Object

Source: <https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>



Classes, Superclasses, and Subclasses

> A subclass inherits from its superclasses:

- >> instance variables (fields)
 - >> methods
- } unless they are private in their superclasses

> As a result, a subclass:

- >> includes the **state** defined in its superclasses (and may add more)
- >> can **respond** to the messages represented by superclass methods

> Class Hierarchy

- >> classes with shared attributes can be grouped together
- >> these groupings form a **class hierarchy**, organised as a tree
- >> the hierarchy expresses an “IS A” (or “is a kind of”) relationship
e.g. *Student IS A Person*

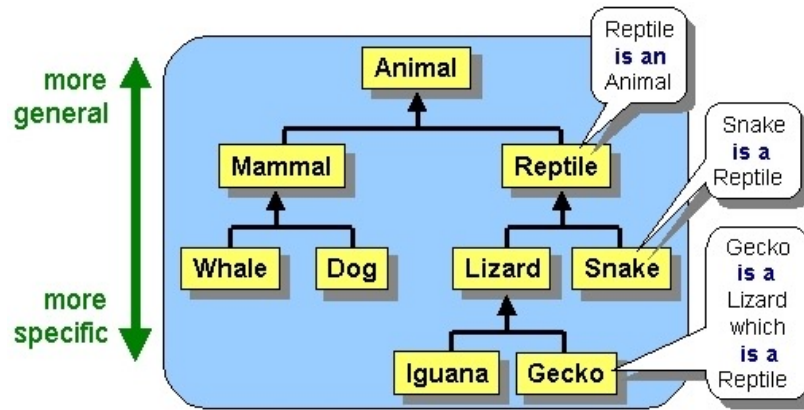
Note that records cannot be extended, and a record cannot extend another class.



Classes, Superclasses, and Subclasses

> Benefits of Inheritance

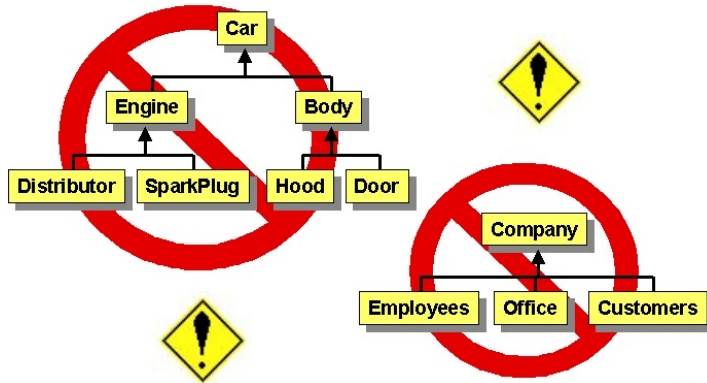
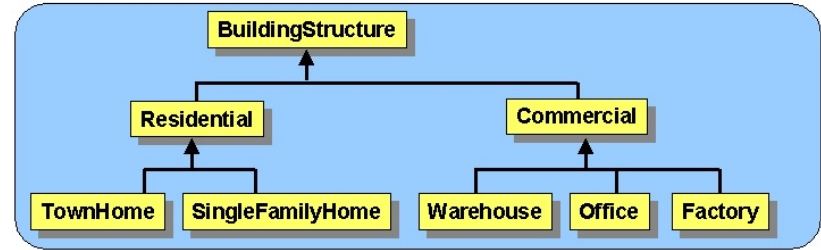
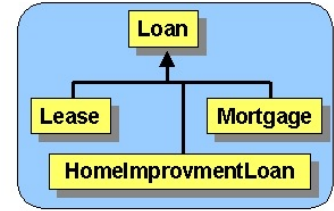
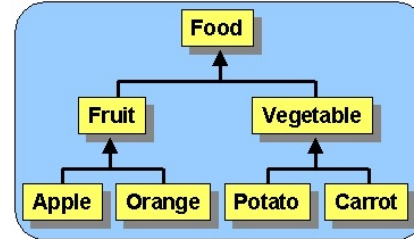
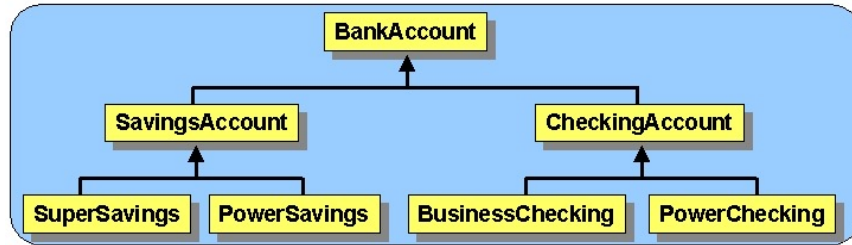
- >> enables **sharing of state and behaviour** from a more general class (superclass)
- >> promotes **code reuse** across related classes (software reusability)
- >> **reduces development time** (less code to be re-written)
- >> encourages reuse of well-designed objects (proven and tested)
- >> helps keep programs structured, readable, and easier to maintain



Source: <http://people.scs.carleton.ca/~lalonde/105WebPage/Notes/105Notes9.html>



Classes, Superclasses, and Subclasses



is-a relationship vs has-a relationship

Source: <http://people.scs.carleton.ca/~lalonde/105WebPage/Notes/105Notes9.html>

Defining Subclasses

> Scenario (similar to last lecture)

- >> in a company, managers are treated differently from other employees
- >> **both** employees and managers **are paid a salary**
- >>> additionally, **managers receive bonuses** when they achieve specific goals

> Design insights

- >> a *Manager* shares most properties and behaviour with an *Employee*
- >> only **additional functionality is needed** (e.g., bonuses)
- >>> Manager **IS-A** Employee

```
% public class Manager extends Employee {  
    //added methods and fields  
}
```

← **extends** indicates that the new class “Manager” derives from the existing class “Employee”

Which class provides more functionality: the Manager subclass or the Employee superclass?



Defining Subclasses

```
% public class Manager extends Employee {  
    private double bonus;  
    . . .  
    public void setBonus(double bonus) {  
        this.bonus = bonus;  
    }  
}
```

Manager class has a new field to store the bonus,
and a new method to set it.

> We can create a Manager and set a bonus:

```
% Manager boss = new Manager(...);  
% boss.setBonus(5000);
```

Can we do the same to an employee?
`empl.setBonus(1000);`



Defining Subclasses

- > Assuming that employees have:
- >> private instance fields: name, salary, hireDay
- >> public methods: getName, getHireDay, ...

```
% Manager boss = new Manager(...);  
% boss.getName();  
% boss.getHireDay(); } from superclass  
  
% boss.salary = 1000; } not allowed, salary is private
```



Overriding Methods

- > The salary of the Manager is not calculated as the salary of the Employee
- >> Employee has a `getSalary()` method
- >> We need to override that method for the Manager to include the bonus

```
% public class Manager extends Employee {  
    . . .  
    public double getSalary() {  
        return super.getSalary() + bonus;  
    }  
}
```

→ is that correct?

↓
no, we need to use the public interface (`getSalary()` method)

← `getSalary()` calls itself

we must use `super.getSalary()` to indicate we want to call the `getSalary()` method from the Employee superclass. **super** is a special keyword that directs the compiler to invoke the superclass method



Subclass constructors

> The Manager constructor:

```
% public Manager(String name, double salary, int year, int month, int day) {  
    super(name, salary, year, month, day);  
    bonus = 0;  
}
```

← super calls the constructor
of the superclass

Why do we call the superclass constructor instead of initializing the fields directly in the Manager constructor?

→ *The Manager constructor cannot access the private fields of the Employee class*

What happens if the constructor of the superclass is not explicitly called?

→ *The superclass must have a no-argument constructor, which is invoked before the subclass is constructed*



< Code time >

> Create one manager and two employees, then print their details using an array and inheritance

```
% for ([which type goes here: Employee or Manager] e : staff)
    IO.println(e.getName() + " " + e.getSalary()); //which salary is printed?
```

When `e` refers to an `Employee` object, the call `e.getSalary()` calls the `getSalary` method of the `Employee` class.

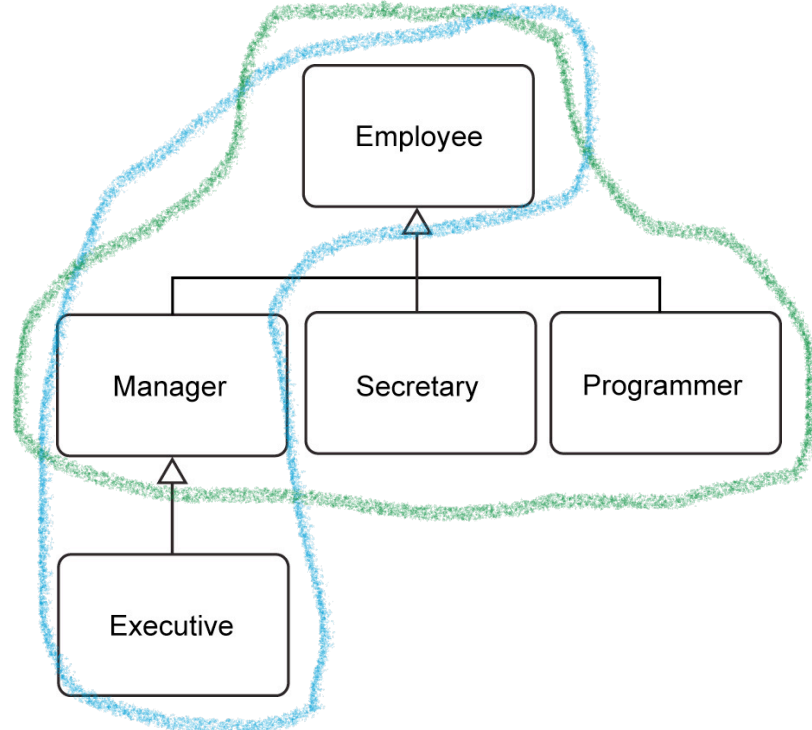
When `e` refers to a `Manager` object, then the `getSalary` method of the `Manager` class is called instead.

The **virtual machine** knows about the actual type of the object to which `e` refers, and therefore can invoke the correct method.



Inheritance Hierarchies

- > Inheritance is not limited to a single level
 - >> a class can extend another subclass
 - >>> *Executive extends Manager, which extends Employee*
- > A superclass may have many subclasses
 - >> *Manager, Programmer and Secretary extend Employee*
 - >> share common behaviour with Employee
 - >>> but are otherwise independent of each other
- > Java supports single inheritance
 - >> a class may only extend only one direct superclass
 - >> *Python and C++ supports multiple inheritance*



Employee inheritance hierarchy



<Exercises>

What does a subclass inherit?

- A. Only public methods
- B. Only fields
- C. Fields and accessible methods
- D. Constructors



<Exercises>

Which statement best describes inheritance?

- A. A subclass copies the superclass code
- B. A subclass is a specialised version of its superclass
- C. A superclass depends on its subclass
- D. Inheritance prevents polymorphism



<Exercises>

Why must `super(...)` be called in a subclass constructor?

- A. To override superclass methods
- B. To initialise inherited fields properly
- C. To prevent polymorphism
- D. It is optional and unnecessary



<Exercises>

What happens if a superclass constructor is not explicitly called?

- A. The program crashes
- B. The fields remain uninitialised
- C. The subclass constructor is ignored
- D. The JVM inserts a call to the no-argument constructor



<Exercises>

In Java, what is the root class from which all classes are derived?

- A. java.lang.Root
- B. java.lang.Class
- C. java.lang.Object
- D. java.lang.Base



<Exercises>

Which relationship does inheritance properly represent?

- A. HAS-A relationship
- B. USES-A relationship
- C. IS-A relationship (or "is a kind of")
- D. WORKS-WITH relationship



<Exercises>

When a subclass defines a method with the same signature as a method in its superclass, this is called:

- A. Overloading
- B. Overriding
- C. Hiding
- D. Duplication



Polymorphism

> We have just seen polymorphic behaviour, even though you did not yet know what it was called
>> so, **what is polymorphism?**

> *"Poly" many; "morphism" forms*

>> *the ability of an object to take on different forms*

>> *in Java, it refers to the ability of the same method call to produce different behaviour, depending on the actual object type involved*

How it works?

> A variable can be declared using a **superclass** or **interface** type

> At runtime, the method implementation of the actual object is executed

interfaces will be covered later

the reference type decides what methods are allowed;
the object type decides which implementation runs



Polymorphism

Violating the Substitution Principle: If `e2.getSalary()` [Manager] promises to always return a non-negative value, but it (as a subclass) overrides it to sometimes throw an exception or return something invalid, then it (the subclass) cannot safely substitute the superclass (Employee).

How it works?

- > A variable can be declared using a **superclass** or **interface** type
- > At runtime, the method implementation of the actual object is executed
- > Achieved through inheritance and method overriding
- > Method calls are dynamically bound at runtime

```
% Employee e1 = new Employee();  
% Employee e2 = new Manager();  
% e1.getSalary();  
% e2.getSalary();
```

Although both variables are of type Employee, the method invoked depends on the **object they refer to**

- > *polymorphism* supports substitutability (a subclass can be used wherever its superclass is expected, without changing the correctness of the program) [Liskov Substitution Principle]
- > *polymorphism* reduces coupling and simplifies code that works with families of related objects



Polymorphism

Example 1:

```
% class Animal {  
    void makeSound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
Animal animal1 = new Dog();  
Animal animal2 = new Cat();  
  
animal1.makeSound();    // Dog barks  
animal2.makeSound();    // Cat meows
```



Polymorphism

Example 2:

```
% class Device {  
    void turnOn() {  
        System.out.println("Turning on device");  
    }  
}  
  
class TV extends Device {  
    @Override  
    void turnOn() {  
        System.out.println("TV is now on");  
    }  
}  
  
class Projector extends Device {  
    @Override  
    void turnOn() {  
        System.out.println("Projector warming up");  
    }  
}
```

```
Device device1 = new TV();  
Device device2 = new Projector();  
  
device1.turnOn();    // TV is now on  
device2.turnOn();    // Projector warming up
```



Polymorphism

Why is using a switch statement discouraged in object-oriented programming?

> In procedural languages, polymorphism is typically implemented using a switch statement with multiple cases:

```
% switch (employeeType) {  
    case MANAGER:  
        return salary + bonus;  
    case CEO:  
        return (salary + bonus) * someRate;  
    default:  
        return salary;  
}
```

```
% public enum EmployeeType {  
    EMPLOYEE,  
    MANAGER,  
    CEO  
}
```

often a smell that polymorphism might be a better solution



<Exercises>

Using polymorphism, implement a better solution for the following code:

```
% void draw(Shape shape) {  
    switch (shape.getType()) {  
        case CIRCLE -> drawCircle(shape);  
        case RECTANGLE -> drawRectangle(shape);  
    }  
}
```



<Answer>

```
% class Shape {  
    void draw() {  
        IO.println("Drawing a shape");  
    }  
}
```

```
@Override  
class Circle extends Shape {  
    void draw() { IO.println("Drawing a circle"); }  
}
```

```
@Override  
class Rectangle extends Shape {  
    void draw() { IO.println("Drawing a rectangle"); }  
}
```

```
Shape s1 = new Circle();  
Shape s2 = new Rectangle();  
  
s1.draw();    // Drawing a circle  
s2.draw();    // Drawing a rectangle
```



Polymorphism

Caution!

> In Java, arrays of subclass references can be converted to arrays of superclass references without a cast. For example, consider the following:

```
% Manager[] managers = new Manager[10];  
% Employee[] staff = managers;
```

```
% staff[0] = new Employee("Harry Hacker", . . .);
```

> `staff[0]` and `managers[0]` are the same reference, so it looks as if we managed to smuggle a mere employee into the management ranks

> `managers[0].setBonus(1000)` would try to access a non-existent instance field and would corrupt neighbouring memory

> Attempting to store an Employee reference causes an **ArrayStoreException**



Understanding Method Calls

- > To understand **polymorphism** and **dynamic binding**, it is essential to understand how Java resolves and executes a method call
- > Although a method call may appear simple in source code, it involves both compile-time and runtime decisions

- > Consider a method call of the form:

```
% x.f(args);
```

the implicit argument x is declared to be of type C

We saw earlier that methods can have the same name but different parameter types

- > **Method selection at compile time (step 1)**

- > the compiler begins by examining:

- >> the **declared type of the object**: x (class C) and the **method name** as there might be many methods with the same name (in our case, f(int), f(String), f(Employee))

- >>> the **compiler collects all accessible methods** named f that are defined **in class C, and inherited from its superclasses** (excluding private superclass methods, which are not accessible)

At this point, the compiler has a set of all candidate methods to be called



Understanding Method Calls

> Overloading Resolution (step 2)

> the compiler considers the types of the arguments supplied in the call

>> if there exists a **single best-matching method** among the candidates, that method is selected

> Consider the following method call:

```
% x.f("Hello");
```

and the following methods: `f(int)` and `f(String)`

← Which method does the compiler select?

type conversions (e.g. `int` → `double`, `Manager` → `Employee`)

The drama is if...

No method matches the argument types, OR multiple methods match equally well after applying conversions

↖ in this case, the compiler reports an error

At the end of this step, the compiler knows the method (name + parameter types) that must be invoked



(Let me open a parenthesis for a moment)

← this is called method overloading

- > We know that `f(int)` and `f(String)` are two methods with the same name but different signatures!
- > We also know that if a method in a subclass has the same signature as a method in the superclass, the subclass method overrides the superclass method
- > By now, we should also know that the **return type is not part of the signature**

However, when you override a method, you need to keep the return type compatible

Why did we open a parenthesis if we already know all of this?

... because a subclass may change the return type to a subtype of the original type

For example, suppose the `Employee` class has a method

```
% public Employee getBuddy() {...}
```

```
% public Manager getBuddy() {...}
```

...assuming from our previous examples that
`Manager` "is-a" `Employee`

- We say that the two `getBuddy` methods have covariant return types:
 - > if `B` is a subtype of `A`, then a covariant context allows `B` to be used where `A` is expected



Understanding Method Calls

> Static vs Dynamic Binding (step 3)

> if the method is: private, static, final, or a constructor

>> then the compiler knows (**static binding**) exactly which method to call

>> **Private methods** are not visible to subclasses (cannot be overridden)

>> **Static methods** belong to the class, not to objects (method selection depends on the declared type, not the runtime object)

>> **Final methods** are explicitly declared as not overridable (no subclass can provide a different implementation)

>> **Constructors** are never inherited or overridden (each new call refers to a specific constructor)

> Otherwise, the method call is resolved at runtime (**dynamic binding**), based on the actual object type

>> this is the **foundation of polymorphism**



Understanding Method Calls

> Method Dispatch at Runtime (step 4)

> When dynamic binding is used, the JVM must determine which method implementation to execute

>> Let the actual runtime type of `x` be `D`, a subclass of `C`, the JVM:

>>> checks whether class `D` defines the method

>>> if not, searches the superclass of `D`

>>> continues up the inheritance chain until a matching method is found

To avoid repeating method searches on every call, the JVM uses method tables. Each class has a precomputed table that maps method signatures to their concrete implementations. This table is built when the class is loaded, by combining the methods defined in the class with the method table of its superclass. As a result, method invocation at runtime is reduced to a fast table lookup. If a method is invoked using `super.f(args)`, the JVM consults the method table of the superclass directly.



Understanding Method Calls

Example (*ignoring the Object superclass*)

Employee (method table)

```
getName() -> Employee.getName()  
getSalary() -> Employee.getSalary()  
getHireDay() -> Employee.getHireDay()  
raiseSalary(double) -> Employee.raiseSalary(double)
```

...it is a single method, and therefore no overloading occurs; it is not private, static, final, or a constructor, so it is dynamically bound

Manager (method table)

```
getName() -> Employee.getName()  
getSalary() -> Manager.getSalary()  
getHireDay() -> Employee.getHireDay()  
raiseSalary(double) -> Employee.raiseSalary(double)  
setBonus(double) -> Manager.setBonus(double)
```

Three methods are **inherited**, one method is **redefined**, and one method is **added**

How `e.getSalary()` is resolved (if `e` is a `Manager`)?



Preventing Inheritance: Final Classes and Methods

```
% public final class Executive extends Manager {  
    ...  
}
```

> Classes that cannot be extended are called *final* classes, and the *final modifier* is used in the definition of the class to indicate this

```
% public class Employee {  
    public final String getName() {  
        return name;  
    }  
}
```

- > individual methods can also be declared final
- > a final method **cannot** be overridden by subclasses
- > all methods in a final class are automatically final

Why make a Class or Method Final?

To ensure that its semantics cannot be changed by subclasses

Examples:

String is final → guarantees immutability and correctness
(you know it refers to a String and nothing but a String)

Calendar.getTime() is final → preserves internal consistency

Final Fields vs Final Classes

- > a final field cannot be reassigned after construction
- > a final class does not make fields final automatically
- > only methods become implicitly final in a final class



Preventing Inheritance: Final Classes and Methods

```
% public class Executive extends Manager {  
    private String title;  
  
    public Executive(String name, String title, double salary,  
        int year, int month, int day) {  
        super(name, salary, year, month, day);  
        this.title = title;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public String description() {  
        if (title.length() >= 20)  
            return "An executive with an impressive title";  
        else  
            return "An executive with a title of " + title;  
    }  
}
```

Problem Example: Overriding in Constructors

- > Constructors execute **top-down** through the inheritance chain
- > Method calls are **polymorphic**
- > Subclass state may not yet be initialized
- >> when Executive is constructed, its constructor first calls the Manager constructor, which calls the Employee constructor, which calls the description method (title is null as Executive constructor have not finished yet).

Result

Overridden method executes too early (unsafe in constructors)
Can cause **NullPointerException**



<Exercises>

What determines which overridden method is executed?

- A. The declared type of the variable
- B. The return type
- C. The runtime type of the object
- D. The parameter names



<Exercises>

What principle allows a subclass to be used wherever its superclass is expected?

- A. Encapsulation Principle
- B. Open–Closed Principle
- C. DRY Principle
- D. Liskov Substitution Principle



<Exercises>

In Java, polymorphism is achieved through:

- A. Inheritance and method overloading
- B. Inheritance and method overriding
- C. Encapsulation and data hiding
- D. Abstract classes only



<Exercises>

Given:

```
Employee e = new Manager();  
e.getSalary();
```

Which class's `getSalary()` method is executed at runtime?

- A. The Employee class's method
- B. The Manager class's method (if overridden)
- C. Both methods execute
- D. It causes a compilation error



Casting

> Recall: converting one type to another is called *casting*

```
% double x = 3.405;  
% int nx = (int) x;
```

← converts the value of the expression x into an integer, discarding the fractional part

> We can also convert an object reference from one class to another

```
% var staff = new Employee[3];  
% staff[0] = new Manager("Carl Cracker", 80000, 1987, 12, 15);  
% staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);  
% staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);  
  
% Manager boss = (Manager) staff[0];
```

There is only one reason why you would want to make a cast: to **use an object in its full capacity after its actual type has been temporarily forgotten**



Casting

> Recall: converting one type to another is called *casting*

% double x = 3.405; _____ converts the value of the expression x into an integer,

% i Note that the compiler checks that you do not promise too much when you store a value in a variable.

> W

% v if you assign a subclass reference to a superclass variable, you are promising less, and
% s the compiler will simply let you do it.

% s

% s if you assign a superclass reference to a subclass variable, you are promising more.
% M Then you must use a cast so that your promise can be checked at runtime.

There is only one reason why you would want to make a cast: to use an object in its full capacity after its actual type has been temporarily forgotten



Casting

> What happens if a downcast does not match the object's runtime type?

```
% var staff = new Employee[3];  
% staff[0] = new Manager("Carl Cracker", 80000, 1987, 12, 15);  
% staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);  
% staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
```

```
% Manager boss = (Manager) staff[1];
```

Java runtime system notices the broken promise and generates a `ClassCastException`

> Can we safely check whether a cast will succeed before performing it?

```
% if (staff[i] instanceof Manager) {  
    boss = (Manager) staff[i];  
    ...  
}
```

You can cast only within an inheritance hierarchy. Note that `x instanceof C` returns false if `x` is null; it does not throw an exception, since null refers to no object.

`staff[i].getSalary()` invokes the correct method through polymorphism; a cast is only required to call methods specific to `Manager` (such as `setBonus`). Alternatively, you may reconsider the class design and declare `setBonus` in the `Employee` superclass.



Pattern Matching for *instanceof*

Pattern matching is a language feature that lets you test a value's type or structure and, if the test succeeds, automatically extract it into a variable (all in one step).

> We can use pattern matching instead of:

> Traditional *instanceof*

```
% if (staff[i] instanceof Manager) {  
    Manager boss = (Manager) staff[i];  
    boss.setBonus(5000);  
}
```

> if `staff[i]` is an instance of the `Manager` class, then the variable `boss` is set to `staff[i]`, as a `Manager`. Cast is skipped.

> Pattern Matching

```
% if (staff[i] instanceof Manager boss) {  
    boss.setBonus(5000);  
}
```



Pattern Matching for *instanceof*

> A useless `instanceof` pattern is a compile-time error:

```
% Manager boss = . . . ;  
    if (boss instanceof Employee e) ... // ERROR: Of course it's an Employee
```

> **The condition is always true** as `boss` is declared as `Manager` and `Manager` extends `Employee`

> Every `Manager` is already an `Employee`: there is no runtime uncertainty, no possible alternative outcome, no real “test” being performed

> With pattern matching (`instanceof T v`), Java enforces a stronger rule:

>> the test must be **meaningful** and must be able to either succeed or fail at runtime

>> if the test can **never fail**, the pattern is considered **useless**, and Java reports it as a **compile-time error**

>>> this is intentional: it prevents misleading or pointless code

```
% Employee e = ... ;  
if (e instanceof Manager m) { // This is OK, why?  
    m.setBonus(1000);  
}
```

For more on pattern matching, read Section 5.1.9 in Core Java, Volume I; also read 5.1.10 on Protected Access



Object: The Cosmic Superclass

- > The `Object` class is the **ultimate superclass** in Java
- > Every class in Java implicitly extends `Object`

`% public class Employee extends Object` ← implicit declaration
if no superclass is specified, `Object` is assumed by default

Why do we need to know that every class extends `Object`? How does this impact my own classes?
Because all classes inherit from `Object`, its methods and services are available to every Java object!

Examples

> `toString()`

>> Returns a string representation of the object

> `equals(Object obj)`

>> Compares this object with another for logical equality

> `hashCode()`

>> Returns a hash code value for the object (used in hash-based collections)



Object: The Cosmic Superclass

> We can also declare “Variables of Type Object”

```
% Object obj = new Employee("Harry Hacker", 35000);  
% Object objArr = new int[10]; // arrays of primitives or objects
```

>> a variable of type Object is only useful as a generic holder for arbitrary values

>> as we have seen before, we can cast to do anything specific (e.g., `Employee e = (Employee) obj;`)

> The equals Method

>> The equals method (in the Object class) determines whether two object references are identical

```
% public boolean equals(Object obj) {  
    return this == obj;  
}
```

—————> However, it is often desirable to implement state-based equality, whereby two objects are considered equal if they share the same state

```
% class A {}  
% var a = new A();  
% var a1 = a;  
% a.equals(a1);
```



Object: The Cosmic Superclass

```
% public class Employee {    ...
    public boolean equals(Object otherObject) {
        // a quick test to see if the objects are identical
        if (this == otherObject) return true;

        // must return false if the explicit parameter is null
        if (otherObject == null) return false;

        // if the classes do not match, they cannot be equal
        if (getClass() != otherObject.getClass()) return false;

        // now we know otherObject is a non-null Employee
        Employee other = (Employee) otherObject;

        // test whether the fields have identical values
        return name.equals(other.name)
            && salary == other.salary
            && hireDay.equals(other.hireDay);
    }
}
```



Object: The Cosmic Superclass

```
% public class Employee {    ...  
    public boolean equals(Object otherObject) { ...  
        // test whether the fields have identical values  
        return name.equals(other.name)  
            && salary == other.salary  
            && hireDay.equals(other.hireDay);  
    }  
}
```

To guard against the possibility that name or hireDay are null, use the `Objects.equals` method. The value of `Objects.equals(a, b)` is true if both arguments are null, false if only one is null, and `a.equals(b)` otherwise.

> to prevent `name.equals(...)` or `hireDay.equals(...)` from being invoked on a null reference:

```
% return Objects.equals(name, other.name)  
    && salary == other.salary  
    && Objects.equals(hireDay, other.hireDay);
```



Object: The Cosmic Superclass

> When implementing equals in a subclass, `super.equals(...)` should be called to include the superclass state in the equality check

```
% public class Manager extends Employee {  
    . . .  
    public boolean equals(Object otherObject) {  
        if (!super.equals(otherObject)) return false;  
        // super.equals checked that this and otherObject belong to the same class  
        // if the superclass fields are equal,  
        // then compare the instance fields of the subclass  
        Manager other = (Manager) otherObject;  
        return bonus == other.bonus;  
    }  
}
```



Object: The Cosmic Superclass

> The Java Language Specification **requires** that the **equals** method has the following properties:

1. It is reflexive:

> For any non-null reference `x`, `x.equals(x)` should return true

2. It is symmetric:

> For any references `x` and `y`, `x.equals(y)` should return true if and only if `y.equals(x)` returns true.

3. It is transitive:

> For any references `x`, `y`, and `z`, if `x.equals(y)` returns true and `y.equals(z)` returns true, then `x.equals(z)` should return true

4. It is consistent:

> If the objects to which `x` and `y` refer haven't changed, then repeated calls to `x.equals(y)` return the same value

5. For any non-null reference `x`, `x.equals(null)` should return false

see documentation for equals()



Object: The Cosmic Superclass

> How should the `equals` method behave if the implicit and explicit parameters do not belong to the same class?

>> We have seen that `equals` returns false if the classes do not match exactly (when using `getClass()`)
>> however, many programmers use an `instanceof` test instead:

```
% if (!(otherObject instanceof Employee)) return false;
```



it may be considered equal because it belongs to the same type hierarchy and satisfies the equality conditions defined in `Employee`

> **Revisiting the symmetry rule**

>> if `e.equals(m)` is true, then `m.equals(e)` must also be true

>> if `Employee.equals` uses an `instanceof` test, then `e.equals(m)` may return true

>> **But under what conditions should a `Manager` be equal to an `Employee`?**

>>> the `Manager.equals` method would need to compare itself with any `Employee` without considering manager-specific state



Object: The Cosmic Superclass

> There are two distinct design scenarios

>> if subclasses define their own notion of equality, the **symmetry** requirement necessitates the use of a `getClass()` test to ensure that only objects of the exact same class are compared

>> if the notion of equality is fixed in the superclass and is not extended or refined by subclasses, an `instanceof` test may be used, allowing objects of different subclasses to be considered equal

Suppose equality is based on a unique employee ID. This definition of equality is meaningful and consistent across all subclasses. In such a case, an `instanceof` test may be used, and `Employee.equals` should be declared `final` to prevent subclasses from altering the equality semantics.

Note that the standard **Java library contains over 150 implementations of `equals`**, reflecting a variety of **design choices**, including both `instanceof` tests and `getClass()` comparisons.



Object: The Cosmic Superclass

Recommended steps for implementing equals

1. Name the parameter `otherObject`. It will later be cast to a variable named `other`
2. Check [reference](#) equality
`% if (this == otherObject) return true; // This may improve performance for classes with complex state`
3. Check for null (required):
`% if (otherObject == null) return false;`
4. Check type compatibility:
> If equality semantics may vary in subclasses, use `getClass()`:
`% if (getClass() != otherObject.getClass()) return false;`
`% ClassName other = (ClassName) otherObject;`
> if equality semantics are consistent across subclasses, use `instanceof`:
`% if (!(otherObject instanceof ClassName other)) return false;`
5. Compare fields according to the equality definition: (a) use `==` for primitive fields; or, (b) [Objects.equals\(\)](#) for object fields
> if overriding in a subclass, include `super.equals(other)`





```
public class Employee {  
    private String name; 2 usages  
    private LocalDate hireDay; 2 usages  
    private double salary; 2 usages  
    public boolean equals(Employee other) { 4 usages  
        return other != null  
            && getClass() == other.getClass()  
            && Objects.equals(name, other.name)  
            && salary == other.salary  
            && Objects.equals(hireDay, other.hireDay);  
    }  
}
```



Object: The Cosmic Superclass

> The hashCode Method

- >> returns an int value representing the object's hash code
- >> the default implementation typically derives a value from the object's memory identity
- >> two distinct objects usually produce different hash codes (not guaranteed)

> The equals–hashCode Contract

- >> if a class overrides `equals`, it must also override `hashCode`
- >> the contract states
 - >>> if `x.equals(y)` is true, then `x.hashCode() == y.hashCode()` must also be true
 - >>> if `x.equals(y)` is false, their hash codes may or may not differ
 - >>> the hash code must be consistent during an object's lifetime (provided fields used in equality do not change)



Object: The Cosmic Superclass

> The String class uses the following algorithm to compute the hash code:

```
% int hash = 0;  
% for (int i = 0; i < length(); i++)  
    hash = 31 * hash + charAt(i);
```

✦ 31 is used in hash code calculations because it is prime, efficient to compute, and provides good hash distribution.

> the hashCode method is defined in the Object class.

>> every object has a default hash code, called the **identity hash code**

```
% var s = "Ok";  
% var sb = new StringBuilder(s);  
% IO.println(s.hashCode() + " " + sb.hashCode());  
% var t = new String("Ok");  
% var tb = new StringBuilder(t);  
% IO.println(t.hashCode() + " " + tb.hashCode());
```

> s and t have the same hash code because String computes hash codes based on content
> sb and tb have different hash codes because StringBuilder does not override hashCode; it inherits the default implementation from Object (typically based on object identity)

Object	Hash Code	Object	Hash Code
s	2556	t	2556
sb	20526976	tb	20527144

Hash Codes of Strings and String Builders



Object: The Cosmic Superclass

> Implementing `hashCode()` in `Employee`

```
% public int hashCode() {  
    return 7 * Objects.hashCode(name)  
        + 11 * Double.hashCode(salary)  
        + 13 * Objects.hashCode(hireDay);  
}
```

> Use distinct multipliers to reduce collisions, especially when fields have limited value ranges.

> Alternatively, use the utility method:

```
% public int hashCode() {  
    return Objects.hash(name, salary, hireDay);  
}
```

> For arrays: use `Arrays.hashCode(...)`; for multi-dimensional arrays: use `Arrays.deepHashCode(...)`

`equals` and `hashCode` must be consistent: if equality is based on *employee ID*, the hash code must also be computed from the ID



Object: The Cosmic Superclass

> The toString Method

- >> the toString method returns a string representation of this object
- >> commonly used for debugging and logging
- >> automatically called when an object is printed (e.g., IO.println(obj))
- >> most classes override toString() to provide a more meaningful, human-readable description of the object's state

Example

// Employee Class

```
% public String toString() {  
    return getClass().getName()  
        + "[name=" + name  
        + ",salary=" + salary  
        + ",hireDay=" + hireDay  
        + "];"  
}
```

subclass

// Manager Class (extends Employee Class)

```
% public String toString() {  
    return super.toString()  
        + "[bonus=" + bonus  
        + "];"  
}
```

The call to toString() can be omitted; for example, IO.println(boss) implicitly invokes boss.toString().
Use Arrays.toString(array) to print an array, or Arrays.deepToString(array) for multidimensional arrays.



<Exercises>

When is casting required?

- A. When assigning subclass reference to superclass variable
- B. When assigning superclass reference to subclass variable
- C. When calling overridden methods
- D. When using polymorphism



<Exercises>

If a class overrides equals, it must also override:

- A. hashCode
- B. toString
- C. clone
- D. compareTo



<Exercises>

What happens to methods in a final class?

- A. They become private
- B. They are automatically final as well
- C. They can still be overridden
- D. They become static



<Exercises>

Why is the String class declared final in Java?

- A. To save memory
- B. To guarantee immutability and correctness
- C. To make it faster
- D. To prevent instantiation



<Exercises>

What is the only reason to use a cast with object references?

- A. To change the object's type permanently
- B. To create a new object
- C. To convert primitives to objects
- D. To use an object in its full capacity after its actual type has been temporarily forgotten



Generic ArrayLists

> Array vs ArrayList

- >> Once an array's size is set, it cannot be changed
- >> To manage dynamically sized collections, Java provides the `ArrayList` class
- >> `ArrayList` behaves like an array but automatically resizes as elements are added or removed
- >> `ArrayList` is a generic class
- >>> the element type is specified using angle brackets

Declaring ArrayLists

```
% ArrayList<Employee> staff = new ArrayList<Employee>();  
% var staff = new ArrayList<Employee>();  
% ArrayList<Employee> staff = new ArrayList<>();  
% ArrayList<Employee> staff = new ArrayList<>(100);
```

equivalent ways to declare and initialize an `ArrayList`. The diamond (`<>`) syntax infers the type (`Employee`) from the variable declaration

Adding elements to an ArrayList

```
% staff.add(new Employee("John", ...));
```

Size of an ArrayList

```
% staff.size(); //equivalent to arr.length
```

An array created with `new Employee[100]` allocates 100 fixed slots. An `ArrayList` with an initial capacity of 100 can hold up to 100 elements before resizing; beyond that, it reallocates internally.



Generic ArrayLists

> Accessing ArrayList Elements

>> the ArrayList class is not part of the core Java language – it is a utility class provided in the standard library

```
% staff.set(i, john);
```

← equivalent to `array[i] = john;`
indices are zero-based for ArrayLists

← too
values can only be assigned **after they are added**;
use `add` to insert elements and `set` to **replace an existing element** at a given index.

```
% Employee e = staff.get(i);
```

← equivalent to `Employee e = array[i];`

> Transforming an ArrayList into an array

```
% var list = new ArrayList<Employee>();  
while (condition) {  
    x = ...; // create or obtain an Employee  
    list.add(x);  
}
```

→ `var arr = new Employee[list.size()];
list.toArray(arr);`

Note that adding an element inserts a reference to the same object into the list:
[ref → Employee1, ref → Employee2, ref → Employee3]



Generic ArrayLists

> How to add elements in the middle of the array?

```
% int pos = staff.size() / 2;  
% staff.add(pos, elem); // elements at position pos and above are shifted up
```

> How to remove elements from the array?

```
% Employee e = staff.remove(pos); // return the removed elem; size is reduced by one
```

> How to traverse the contents of an ArrayList?

```
% for (Employee e : staff)  
    // do something with e  
  
% for (int i = 0; i < staff.size(); i++) {  
    Employee e = staff.get(i);  
    // do something with e  
}
```



Generic ArrayLists

> Compatibility Between Typed and Raw ArrayList

>> A raw ArrayList (without a type parameter) disables generic type checking

```
% ArrayList a = new ArrayList(); // warning 'raw use of parameterized class ArrayList'
```

>> A typed ArrayList<Employee> can be assigned to a raw ArrayList, but this produces an unchecked warning

>> Mixing raw and typed collections may lead to runtime ClassCastException

>> Raw types exist for backward compatibility and should generally be avoided in modern code

```
// What does this code do? Does it work? How useful is that?
```

```
% ArrayList arr = new ArrayList();
```

```
    arr.add(new Object());
```

```
    arr.add("Hi");
```

```
    arr.add(new Manager("B", 100, 2025, 1, 1));
```

```
for(var elem : arr) IO.println(elem.getClass() + " - " + elem);
```



Object Wrappers and Autoboxing

- > A **wrapper class** in Java is a class that **encapsulates a primitive data type as an object**
- > Java provides a corresponding **wrapper class for each primitive type**, allowing primitive values to be treated as objects when required (for example, when working with collections or generics)
- > Wrapper classes are regular Objects, immutable, and provide utility methods (e.g., parsing, comparison)

> Some primitive types and their corresponding wrapper classes

int → Integer

double → Double

boolean → Boolean

char → Character

... (names are quite obvious)

> Performance Considerations

>> Autoboxing introduces extra object creation, additional memory usage, possible performance overhead in loops:

```
% for (int i = 0; i < 1_000_000; i++)  
    list.add(i); // creates 1,000,000 Integer objects
```



Object Wrappers and Autoboxing

> Autoboxing and Unboxing

>> Autoboxing is the automatic conversion of a primitive value into its corresponding wrapper object by the Java compiler

```
% Integer x = 10;    // int → Integer (autoboxing)
```

>> Unboxing is the automatic conversion of a wrapper object back into its corresponding primitive value

```
% int y = x;        // Integer → int (unboxing)
```

> Why do we need wrappers?

```
% ArrayList<int> list;    // not allowed  
% ArrayList<Integer> list; // allowed  
% list.add(5);
```

← 5 is a primitive int (autoboxed automatically: int → Integer)



Object Wrappers and Autoboxing

> Without automatic conversions, you would need to write:

```
% Integer x = Integer.valueOf(10); // instead of Integer x = 10;  
% int n = list.get(i).intValue(); // instead of int n = list.get(i);
```

> Unboxing example:

```
% Integer x = 10;  
% int y = x; ← equivalent to int y = x.intValue();
```

> Obviously, there are pitfalls! What happens here:

```
% Integer x = null;  
int y = x; ← Java attempts: x.intValue(); // NullPointerException
```

> Automatic boxing and unboxing also takes place in arithmetic expressions:

```
% Integer n = 3;  
% n++;
```

> Wrapper Objects and Equality

```
% Integer a = 1000;  
% Integer b = 1000;  
% System.out.println(a == b);
```

} Always use equals() for wrapper comparison
a = 100, b = 100, a == b (true, cached);
a = 1000, b = 1000, a == b (false, no caching)



Object Wrappers and Autoboxing

> JSHELL TIME

```
% Integer x = 10;    // autoboxing (int → Integer)
% int y = x;         // unboxing (Integer → int)
```

```
% System.out.println(x.getClass()); // class java.lang.Integer
% System.out.println(y);             // What about here?
```

```
% Integer z = null;
% int n = z;    // NullPointerException
```

```
% Integer a = 100;
% Integer b = 100;
% System.out.println(a == b);           // true (cached)
```

```
% Integer c = 1000;
% Integer d = 1000;
% System.out.println(c == d);           // false
% System.out.println(c.equals(d));      // true
```



Methods with a variable number of arguments

- > Java allows methods to accept a variable number of arguments, known as varargs
- > Syntax uses an ellipsis (...) after the type
- > Internally, varargs are treated as arrays

Can you think of any examples where you have seen variable arguments used?

Example

```
% public String formatted(Object... args)
```

```
% "%d".formatted(n)
```

```
% "%d %s".formatted(n, "widgets")
```

← invoke the **same method**, even with different numbers of arguments

- > Object... args is compiled as Object[] args
- > Primitive values are autoboxed (e.g., int → Integer)
- > The method processes args[i] to match format specifiers



Methods with a variable number of arguments

> Defining your own varargs method

```
% public static double max(double... values) {  
    double largest = Double.NEGATIVE_INFINITY;  
    for (double v : values)  
        if (v > largest) largest = v;  
    return largest;  
}
```

% max(3.1, 40.4, -5); ← Compiler translates to: max(new double[] { 3.1, 40.4, -5 });

> Important Properties

>> the varargs parameter is **always the last parameter**

>> it behaves exactly like an array inside the method

>> you may pass:

>>> individual arguments (values separated by commas)

>>> an explicit array of the same type

> the main method may be declared as: `public static void main(String... args)`

Varargs are syntactic sugar for arrays



Abstract Classes

```
% class Circle {  
    private String color;  
    private double radius;  
  
    public Circle(String color, double radius) {  
        this.color = color;  
        this.radius = radius;  
    }  
  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
  
    public double getPerimeter() {  
        return 2 * Math.PI * radius;  
    }  
  
    public void displayColor() {  
        System.out.println("Color: " + color);  
    }  
}
```

← What does this class do?

```
// client  
Circle circle = new Circle("Red", 5.0);  
System.out.println("Area: " + circle.getArea());  
System.out.println("Perimeter: " +  
circle.getPerimeter());
```



Abstract Classes

```
% class Rectangle {  
    private String color;  
    private double length;  
    private double width;  
  
    public Rectangle(String color, double length, double width) {  
        this.color = color;  
        this.length = length;  
        this.width = width;  
    }  
  
    public double getArea() {  
        return length * width;  
    }  
  
    public double getPerimeter() {  
        return 2 * (length + width);  
    }  
  
    public void displayColor() {  
        System.out.println("Color: " + color);  
    }  
}
```

← What does this class do?



Abstract Classes

```
% // client
Circle circle = new Circle("Red", 5.0);
Rectangle rect = new Rectangle("Blue", 4.0, 6.0);

// PROBLEM: Can't create a collection of different shapes
// This won't work!
// Shape[] shapes = {circle, rect}; // No Shape type exists!

// Need separate methods for each shape
printCircleInfo(circle);
printRectangleInfo(rect);

// Or messy instanceof checks with Object array
Object[] objects = {circle, rect};
for (Object obj : objects) {
    if (obj instanceof Circle) {
        Circle c = (Circle) obj;
        System.out.println("Area: " + c.getArea());
    } else if (obj instanceof Rectangle) {
        Rectangle r = (Rectangle) obj;
        System.out.println("Area: " + r.getArea());
    }
}
```

← What if we want to process multiple shapes?



Abstract Classes

```
% class Triangle {
    private String color;
    private double side1;
    private double side2;
    private double side3;

    public Triangle(String color, double s1,
double s2, double s3) {
        this.color = color;
        this.side1 = s1;
        this.side2 = s2;
        this.side3 = s3;
    }

    // DUPLICATION: Same method name but different
implementation
    public double getArea() {
        // Heron's formula
        double s = (side1 + side2 + side3) / 2;
        return Math.sqrt(s * (s - side1) * (s -
side2) * (s - side3));
    }
}
```

What does this class do?

What is a better solution?

```
// DUPLICATION: Same method name but different
implementation
    public double getPerimeter() {
        return side1 + side2 + side3;
    }

    // DUPLICATION: This method is identical to Circle
and Rectangle!
    public void displayColor() {
        System.out.println("Color: " + color);
    }
}
```



Abstract Classes

```
% abstract class Shape {  
    protected String color; // Common field  
  
    public Shape(String color) {  
        this.color = color;  
    }  
  
    public abstract double getArea();  
    public abstract double getPerimeter();  
  
    // Concrete method - shared by all shapes  
    public void displayColor() {  
        System.out.println("Color: " + color);  
    }  
}
```

What does this class do?

the abstract class captures the common pattern

What do these methods do?

in this case, each shape has an area and a perimeter, but they are calculated differently

abstract methods allow subclasses to implement these calculations in their own way

What about the displayColor() method?

this method is shared by all shapes and can be implemented as a concrete method



Abstract Classes

- > An abstract class defines a **common blueprint** for related subclasses
- > model a general **concept** that **should not exist on its own**
- > enforce a **contract** on subclasses
- > **avoid code duplication**

Examples

- > A **Shape** exists conceptually, but you never create a generic Shape. You create a Circle, Rectangle, etc.
- > A **Vehicle** exists conceptually, but you never create a generic Vehicle. You create a Car, Truck, Bicycle, etc.
- > An **Employee** exists conceptually, but you **rarely** create a generic Employee. You create a Manager, Programmer, Secretary, etc.



Abstract Classes

```
% // Circle extends Shape
class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}
```

← What does this class do?

no need to redefine color (use from superclass)
implement the abstract methods (concrete methods)
no displayColor() method (less duplicated code)



Abstract Classes

- > Abstract classes
 - >> cannot be instantiated
 - >> may contain abstract methods (methods without implementation)
 - >> may also contain concrete methods, fields, and constructors

- > An abstract method
 - >> has no body
 - >> must be implemented by subclasses
 - >> cannot be private, final, static

if a class does not implement all abstract methods, it must also be declared abstract

- > Abstract classes enable runtime polymorphism

```
% Shape s = new Circle(5);  
System.out.println(s.area());
```

Abstract classes define what must be done; subclasses define how it is done.



Enum Class

> A little more about enums, now that we have learned other concepts

```
% public enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE }
```

>> enumerations can have methods

>> the type defined by an enum declaration is actually a class (e.g., Size is a class)

>> all enumerated types are subclasses of the abstract class Enum

>> they inherit a number of methods from the abstract class. The most useful one is name, which returns the name of the enumerated constant. For example, Size.SMALL.toString() returns the string "SMALL".

>> Each enumerated type has a static values method that returns an array of all values of the enumeration:

```
% Size[] values = Size.values();  
% public enum Size {  
    ...  
    EXTRA_LARGE {  
        public String toString() { return "XL"; } // method toString()  
    };  
}
```

>> The constructor of an enumeration is always private (you can omit the private modifier)



Design hints for inheritance

- > Place common operations and fields in the superclass

- >> example: put the *name* field into the Person class instead of replicating it in the Employee and Student classes

- > Do not use protected fields

- >> anyone can create a subclass and directly access those fields, breaking encapsulation

- >> in Java, all classes in the same package can access protected members, even if they are not subclasses

- >> protected methods can be appropriate when they are intended to be overridden by subclasses but not exposed for general public use

- > Use inheritance to model the “is-a” relationship

- >> inheritance should not be used merely to reuse code

- >> for example, making Contractor extend Employee is problematic if contractors:

- >>> do not have salaries

- >>> are paid hourly

- >>> do not follow the same rules (e.g., raises)

- >> the relationship fails the “is-a” test: a contractor is not a special kind of employee

- >> when the conceptual relationship is incorrect, inheritance introduces more problems than it solves



Design hints for inheritance

> Do not use inheritance unless all inherited methods make sense

Example

```
% class Holiday extends GregorianCalendar { }  
Holiday christmas = new Holiday(...);  
christmas.add(Calendar.DAY_OF_MONTH, 12); // mutable
```

>> if Christmas is December 25, adding 12 days changes the date to January 6, which is no longer Christmas
>> the object is still of type Holiday, but it no longer represents a holiday. That breaks its logical meaning

>> A holiday is a special kind of date, but not all date operations preserve “holiday-ness”

> Do not change the expected behaviour when you override a method

>> The substitution principle applies not only to syntax but, more importantly, to behaviour

The substitution principle (Liskov Substitution Principle) states:
> A subclass object should be usable wherever a superclass object is expected (without breaking the program’s logic)



Design hints for inheritance

- > Do not change the expected behaviour when you override a method
- >> The substitution principle applies not only to syntax but, more importantly, to behaviour

```
% class Bird {  
    public void fly() {  
        System.out.println("Flying");  
    }  
}  
  
class Penguin extends Bird {  
    public void fly() {  
        throw new UnsupportedOperationException();  
    }  
}  
  
Bird b = new Penguin();  
b.fly(); // Runtime failure
```

even though this compiles, the behaviour is broken. A Penguin cannot substitute a Bird if Bird is expected to fly



Design hints for inheritance

> Use polymorphism, not type information

// whenever you see code like:

```
% if (x instanceof Type1)
    action1(x);
else if (x instanceof Type2)
    action2(x);
```

>> if `action1` and `action2` represent the same conceptual operation, define it as a method in a common superclass or interface:

```
% x.action();
```

It is better because...

>> eliminates type checking (`instanceof`)

>> improves readability

>> simplifies maintenance

>> makes adding new types easier (no need to modify existing conditionals)



<Exercises>

What is an abstract class in Java?

- A. A class that cannot have any methods
- B. A class that cannot be instantiated and is designed to be subclassed
- C. A class with only private members
- D. A class that contains only static methods



<Exercises>

Which of the following can an abstract class contain?

(Select all that apply)

- A. Abstract methods
- B. Concrete methods with implementation
- C. Fields
- D. Constructors



<Exercises>

What is the main advantage of ArrayList over arrays?

- A. It's faster
- B. It automatically resizes as elements are added or removed
- C. It can store any type
- D. It uses less memory



<Exercises>

How do you declare an `ArrayList` that can only hold `Employee` objects?

- A. `ArrayList staff = new ArrayList();`
- B. `ArrayList[Employee] staff = new ArrayList();`
- C. `Employee[] staff = new ArrayList();`
- D. `ArrayList<Employee> staff = new ArrayList<Employee>();`



<Exercises>

How do you access an element at index i in an `ArrayList`?

- A. `staff.get(i)`
- B. `staff[i]`
- C. `staff.elementAt(i)`
- D. `staff.retrieve(i)`



<Exercises>

What is a wrapper class in Java?

- A. A class that wraps around other classes to hide them
- B. A class that encapsulates a primitive data type as an object
- C. A class that contains only static methods
- D. A class that cannot be instantiated



<Exercises>

What happens when the following code executes?

```
Integer x = null;  
int y = x;
```

- A. y becomes 0
- B. NullPointerException is thrown
- C. Compilation error
- D. y becomes null



<Exercises>

Why can't you use primitive types like `int` directly with `ArrayList`?

- A. Because `ArrayList` only works with objects, not primitives
- B. Because `int` is not a class
- C. Because of performance reasons
- D. Both A and B



<Exercises>

What is the rule about the position of a varargs parameter?

- A. It must be the first parameter
- B. It must be the last parameter
- C. It can be anywhere
- D. There can be multiple varargs parameters



<Exercises>

What principle does the Penguin/Bird example violate?

- A. Encapsulation
- B. The Liskov Substitution Principle (subclass changes expected behavior)
- C. The Open/Closed Principle
- D. The Single Responsibility Principle



<Exercises>

Why can't a method be both abstract and static?

- A. Because it would cause a naming conflict
- B. Because abstract methods must be private
- C. Because static methods cannot be in abstract classes
- D. Because abstract methods require overriding, and static methods cannot be overridden



MEME



abstract means: This method has no body and must be implemented by a subclass

static means: This method belongs to the class itself, not to instances, and cannot be overridden

conflict: abstract methods require overriding; static methods cannot be overridden (they are hidden, not polymorphic)

Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)

CRICOS PROVIDER CODE: 00120C